

Unit 1. Introduction to Software Testing

What is Software Testing?

- **Software Testing** is a systematic process of evaluating a software application to ensure it works correctly, meets its specified requirements, and is free of defects or security vulnerabilities. The primary goal is to provide objective, independent information about the software's quality and to mitigate the risk of failure.
 - **Software Testing** involves the implementation of a module or part of a system of a product to evaluate all or some of its properties.
 - It Improves quality, reliability, and performance
-

Need for Software Testing

1. **To Detect Defects Early**
Testing helps identify errors, faults, and bugs in the software at an early stage, reducing the cost and effort required to fix them later.
 2. **To Ensure Quality and Reliability**
Software testing ensures that the application performs reliably under expected conditions and delivers a high-quality product to users.
 3. **To Verify Requirements**
Testing confirms that the software meets specified functional and non-functional requirements as defined by the client or user.
 4. **To Improve Security and Safety**
Testing helps detect vulnerabilities, failures, and risks that could lead to security breaches or system crashes.
 5. **To Reduce Maintenance Cost**
Fixing defects during the testing phase is cheaper than correcting them after deployment, thereby lowering long-term maintenance costs.
 6. **To Enhance Customer Satisfaction**
Well-tested software provides a smooth user experience, increasing customer confidence and satisfaction.
-

Evolution of Software Testing

1. **Debugging-Oriented (Early Days):** Testing was informal, tied to debugging, focusing on removing errors found by programmers after development.
2. **Destruction-Oriented (1970s):** A shift to finding ways to break the code and uncover defects, a more proactive error-finding phase.
3. **Evaluation-Oriented (1980s):** Testing became a measurement of quality, moving beyond just breaking code to assessing software quality, with standards like IEEE 829 emerging.

4. **Prevention-Oriented & Quality Assurance (1990s-2000s):** Focus expanded to influencing development processes for better quality (QA) and continuous feedback, integrating testing earlier.
 5. **Automation & Agile Era (2000s-2010s):** Adoption of Agile, XP, DevOps, and CI/CD accelerated testing, making it continuous and automated to keep pace with rapid releases.
 6. **Modern & AI-Driven Era (2010s-Present):** Embracing cloud, AI, machine learning, and specialized tools for predictive testing, intelligent automation, and monitoring in production (Shift-Right), enabling faster, more complex software delivery.
-

Objectives of Software Testing



1. **To Detect Defects**
Identify errors, bugs, and defects in the software before it is released to users.
 2. **To Verify Requirements**
Ensure that the software functions according to functional and non-functional requirements.
 3. **To Validate the Product**
Confirm that the developed software satisfies user needs and business expectations.
 4. **To Ensure Quality and Reliability**
Improve software quality by ensuring consistent and reliable performance under different conditions.
 5. **To Reduce Risk**
Minimize the risk of software failure in production, including financial loss and reputational damage.
 6. **To Improve Performance and Security**
Identify performance bottlenecks and security vulnerabilities.
 7. **To Build Customer Confidence**
Deliver a stable and defect-free product, increasing user trust and satisfaction.
-

Principles of Software Testing

1. Testing Shows the Presence of Defects

Testing can identify defects present in the software, but it **cannot guarantee the absence of defects**. Even after extensive testing, some hidden or rare defects may still exist. The goal of testing is to **reduce risk**, not to prove perfection.

2. Exhaustive Testing is Impossible

Testing every possible input, output, and path is practically impossible, especially for large systems. Therefore, testing must be **selective and strategic**, focusing on:

- Critical functionalities
- High-risk areas
- Frequently used features

3. Early Testing Saves Time and Cost

Testing activities should begin at the **earliest stages** of software development, such as requirement analysis and design. Early detection of defects reduces:

- Rework
 - Development cost
 - Project delays
- This approach is known as **Shift-Left Testing**.

4. Defect Clustering

Defects are not evenly distributed across the system. Typically, a **small number of modules contain the majority of defects**. By identifying and focusing on these defect-prone areas, testing becomes more efficient and effective.

5. Pesticide Paradox

When the same test cases are executed repeatedly, they eventually stop finding new defects. To overcome this:

- test cases should be reviewed and updated
 - new test scenarios should be added
 - exploratory testing should be performed
- This ensures continuous defect detection.

6. Testing is Context-Dependent

There is no one-size-fits-all testing approach. Testing strategies vary depending on:

- type of application (banking, medical, gaming)
 - level of risk
 - regulatory requirements
- For example, safety-critical systems require more rigorous testing than casual applications.

7. Absence-of-Errors Fallacy

Even if software is free from defects, it may still fail if it does not meet user expectations or business needs. Therefore, testing must focus not only on correctness but also on **usability, performance, and relevance**.

Fault, Error, Bug, Failure:

1. Error

An **error** is a **human mistake** made during software development.

- Occurs due to misunderstanding requirements, logic mistakes, or poor design decisions.
- Introduced by developers, analysts, or designers.

✦ *Example:* A developer writes `if (a > b)` instead of `if (a >= b)`.

2. Fault

A **fault** is the **incorrect code or logic** resulting from an error.

- It is a flaw in the program, design, or process.
- Also called a **defect** in some contexts.

✦ *Example:* The incorrect condition written in the code.

3. Bug

A **bug** is an **informal term** used when a fault is discovered during testing.

- Commonly used by testers.
- Represents unexpected or incorrect behaviour.

✦ *Example:* Tester observes wrong output during test execution.

4. Defect

A **defect** is a **deviation from expected behaviour or requirements**.

- Used in quality and testing documentation.
- Can be found during reviews, testing, or inspections.

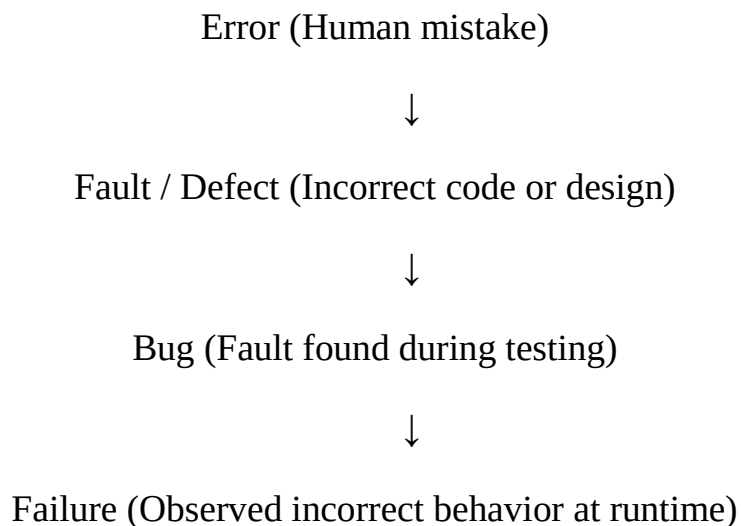
✦ *Example:* Software calculates tax incorrectly compared to requirement document.

5. Failure

A **failure** occurs when the software **does not perform its intended function during execution**.

- Visible to end users.
- Happens when a fault is executed.

✦ *Example:* Application crashes when the user clicks “Submit”.



Testing vs Debugging

Aspect	Testing	Debugging
Primary Goal	Identify and expose errors, failures, and deviations from requirements.	Locate, analyze, and correct the root causes of the identified errors.
Purpose	Verify that the software meets expectations and behaves as intended.	Understand <i>why</i> the software failed and ensure the problem is permanently fixed.

Aspect	Testing	Debugging
Focus	What is wrong with the system?	Why is it wrong, and how can we fix it?
Nature	Evaluation and validation activity	Investigative and corrective activity
Timing	Performed after development or during builds/releases	Performed after a test failure or error is discovered
Output	Failed test cases, error logs, defect reports, or bug tickets	Code modifications, patches, configuration adjustments, or fixes
Responsibility	QA engineers, testers, or automation teams	Developers or programmers
Effect on Code	Does not change the application; it only observes behavior	Directly modifies code or configuration to correct behavior
Goal for Quality	To reveal visible flaws before the software reaches users	To remove underlying issues and ensure they do not recur
End Result	A decision point: pass/fail outcome based on criteria	A functional correction leading to retesting and resolution

Software Testing Life Cycle

Phases of STLC

Requirement Analysis



Test Planning



Test Case Design/Writing



Test Environment Setup



Test Execution



Test Cycle Closure

1. Requirement Analysis:

This is the **first phase** of STLC where testers analyze the software requirements to understand what needs to be tested.

Activities:

- Study functional and non-functional requirements
- Identify testable and non-testable requirements
- Determine testing types required (functional, performance, security, etc.)
- Identify risks and testing priorities
- Interact with stakeholders for clarification

Deliverables:

- Requirement Traceability Matrix (RTM)
- Test feasibility analysis

Importance:

Ensures **complete test coverage** and prevents missing requirements during testing.

2. Test Planning:

In this phase, a **detailed test strategy** is prepared that defines how testing will be carried out.

Activities:

- Define testing scope and objectives
- Choose testing approach and techniques
- Estimate effort, cost, and schedule
- Select tools and resources
- Identify risks and mitigation plans

Deliverables:

- Test Plan document
- Test Strategy
- Resource and effort estimation

Importance:

Acts as a **blueprint for testing activities**.

3. Test Case Design / Development:

Test cases are designed based on requirements and test strategy.

Activities:

- Write test cases and test scenarios
- Prepare test data
- Review and update test cases
- Develop automation scripts (if applicable)

Deliverables:

- Test cases
- Test scripts
- Test data

Importance:

Ensures **systematic and thorough testing** of all functionalities.

4. Test Environment Setup:

A test environment similar to the production environment is prepared.

Activities:

- Configure hardware and software
- Install required tools and applications
- Verify environment readiness using smoke tests

Deliverables:

- Test environment setup report
- Smoke test results

Importance:

Ensures testing is done in a **realistic and stable environment**.

5. Test Execution:

In this phase, test cases are executed and defects are identified.

Activities:

- Execute test cases
- Compare actual results with expected results
- Log defects with severity and priority
- Perform re-testing and regression testing
- Track defect status

Deliverables:

- Defect reports
- Test execution status
- Updated RTM

Importance:

Identifies defects and ensures **functional correctness** of the software.

6. Test Cycle Closure:

This is the **final phase** of STLC where testing activities are formally closed.

Activities:

- Evaluate test completion criteria
- Analyze test metrics (defect density, test coverage)
- Prepare test summary and closure reports
- Document lessons learned

Deliverables:

- Test summary report
- Test closure report
- Lessons learned document

SDLC vs STLC

Aspect	SDLC	STLC
Domain	SDLC is mainly related to software development.	STLC is mainly related to software testing.

Aspect	SDLC	STLC
Focus	Besides development other phases like testing is also included.	It focuses only on testing the software.
Number of Member	In SDLC, more number of members (developers) are required for the whole process.	In STLC, less number of members (testers) are needed.
Team Involved	In SDLC, development team makes the plans and designs based on the requirements.	In STLC, testing team(Test Lead or Test Architect) makes the plans and designs.
Objective	Goal of SDLC is to complete successful development of software.	Goal of STLC is to complete successful testing of software.
End Result	It helps in developing good quality software.	It helps in making the software defects free.
Execution	SDLC phases are completed before the STLC phases.	STLC phases are performed after SDLC phases.
Maintenance	Post deployment support, enhancement, and update are to be included if necessary.	Regression tests are run by QA team to check deployed maintenance code and maintains test cases and automated scripts.
End Result	Creation of reusable software systems is the end result of SDLC.	A tested software system is the end result of STLC.

Software Measurement and Metrics

Software Measurement: A measurement is a manifestation of the size, quantity, amount, or dimension of a particular attribute of a product or process. Software measurement is a titrate impute of a characteristic of a software product or the software process.

Software Measurement Principles

The software measurement process can be characterized by five activities-

1. **Formulation:** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.
2. **Collection:** The mechanism used to accumulate data required to derive the formulated metrics.
3. **Analysis:** The computation of metrics and the application of mathematical tools.
4. **Interpretation:** The evaluation of metrics results in insight into the quality of the representation.
5. **Feedback:** Recommendation derived from the interpretation of product metrics transmitted to the software team.

Need for Software Measurement

Software is measured to:

- Create the quality of the current product or process.
- Anticipate future qualities of the product or process.
- Enhance the quality of a product or process.
- Regulate the state of the project concerning budget and schedule.
- Enable data-driven decision-making in project planning and control.
- Identify bottlenecks and areas for improvement to drive process improvement activities.
- Ensure that industry standards and regulations are followed.
- Give software products and processes a quantitative basis for evaluation.
- Enable the ongoing improvement of software development practices.

Classification of Software Measurement

There are 2 types of software measurement:

1. **Direct Measurement:** In direct measurement, the product, process, or thing is measured directly using a standard scale.
2. **Indirect Measurement:** In indirect measurement, the quantity or quality to be measured is measured using related parameters i.e. by use of reference.

Software Metrics

A metric is a measurement of the level at which any impute belongs to a system product or process.

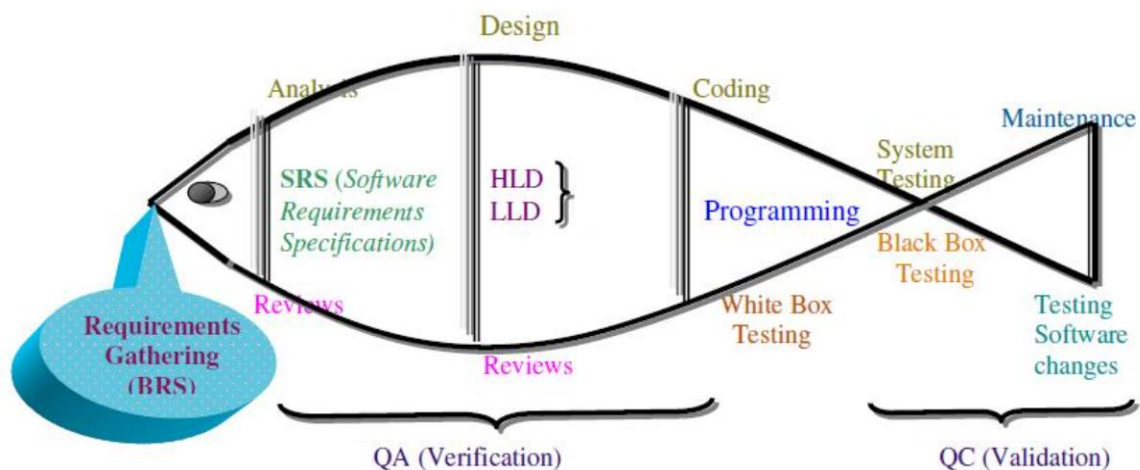
Software metrics are a quantifiable or countable assessment of the attributes of a software product. There are 4 functions related to software metrics:

1. Planning
2. Organizing
3. Controlling
4. Improving

Types of Software Metrics Types of Software Metrics

1. **Product Metrics:** Product metrics are used to evaluate the state of the product, tracing risks and uncover prospective problem areas. The ability of the team to control quality is evaluated. Examples include lines of code, cyclomatic complexity, code coverage, defect density, and code maintainability index.
2. **Process Metrics:** Process metrics pay particular attention to enhancing the long-term process of the team or organization. These metrics are used to optimize the development process and maintenance activities of software. Examples include effort variance, schedule variance, defect injection rate, and lead time.
3. **Project Metrics:** The project metrics describes the characteristic and execution of a project. Examples include effort estimation accuracy, schedule deviation, cost variance, and productivity. Usually measures-
 - Number of software developer
 - Staffing patterns over the life cycle of software
 - Cost and schedule
 - Productivity

Verification Vs Validation



Static V/S Dynamic Testing

Static Testing

- Static testing is an approach to test project documents in the form of Reviews
- BA is going to reviews SRS documents
- Design people will check design & Developer checks their own code.
- This is known as static testing

Dynamic testing

- Dynamic testing is an approach to test the actual software by giving inputs and observing results.
- We Testers are going to check Functionality of Application.
- This is known as Dynamic Testing

Verification	Validation
1. Verification is the static testing.	1. Validation is the dynamic testing.
2. It starts from BRS, Analysis(SRS) will be review, Design & Coding will be reviewed	2. In this we checks/Test whole functionality of software. Maintenance will be done
3. It does not include the execution of the code.	3. It includes the execution of the code.
4. It is known as IN process Testing	4. It is known as OUT process Testing
5. It comes under Quality Assurance (QA)	5. It comes under Quality Control(QC)